

CYNA

Nouvelles fonctionnalités

Explications détaillées

Adrien CACHOUX, Romain CANER, Ethan MIRBEAU

Projet fil rouge - Version 1.1

Solutions de cybersécurité SaaS — SOC · EDR · XDR

Sommaire

Nouvelles fonctionnalités — explications détaillées

Vue d'ensemble

1. Paiement Stripe — pourquoi ça ne marchait pas, et comment c'est réglé
2. Codes de réduction (promotions)
3. Support en temps réel + filtres (application Java)
4. Export CSV des commandes (back-office)

Ce qui a été vérifié (avec Docker)

Ce qui reste à décider

Glossaire

Nouvelles fonctionnalités — explications détaillées

Projet Cyna — Version 1.1 Document destiné à comprendre, pas à pas, les fonctionnalités ajoutées : le paiement Stripe corrigé, les codes de réduction, le support Java en temps réel et l'export CSV des commandes.

Vue d'ensemble

Quatre chantiers ont été menés puis fusionnés dans la branche principale `main` :

#	Fonctionnalité	Où	État
1	Paieement Stripe corrigé	Site web (PHP + JS)	[OK] vérifié en mode simulé
2	Codes de réduction	Site web + back-office	[OK] vérifié de bout en bout
3	Support en temps réel + filtres	Application Java	[OK] compilé et testé
4	Export CSV des commandes	Back-office	[OK] vérifié

Un petit **glossaire** en fin de document explique les termes techniques (PaymentIntent, CSRF, migration...).

1. Paiement Stripe — pourquoi ça ne marchait pas, et comment c'est réglé

Le problème

L'ancienne version envoyait le **numéro de carte complet au serveur Cyna**, qui le transmettait ensuite à Stripe. Pour des raisons de sécurité, **Stripe interdit cette pratique par défaut** : l'appel échouait, donc aucun paiement ne fonctionnait avec de vraies clés.

La solution : Stripe « Payment Element »

La bonne méthode (recommandée par Stripe) inverse la logique : **la carte n'est jamais saisie sur notre serveur**. Elle est tapée dans un composant sécurisé fourni par Stripe (une petite « boîte » affichée dans la page), et c'est le navigateur qui parle directement à Stripe.

Le déroulé exact :

1. Le client arrive sur /checkout.
2. Notre SERVEUR demande à Stripe de créer un « PaymentIntent » (= une intention de paiement pour un montant donné). Stripe renvoie un "client_secret".
3. Le NAVIGATEUR affiche le champ carte de Stripe et, au clic sur payer, confirme la carte directement auprès de Stripe (avec le client_secret).
-> Le numéro de carte ne passe jamais par Cyna.
4. Une fois le paiement accepté, le formulaire de commande part vers notre serveur.
5. Notre SERVEUR revérifie auprès de Stripe que le paiement est bien "succeeded" ET que le montant correspond, avant d'enregistrer la commande.

Le « mode simulé »

Si aucune clé Stripe n'est configurée (par exemple pour une démonstration ou la correction), le système bascule automatiquement en **mode simulé** : le paiement est considéré comme réussi sans débit réel, ce qui permet de tester tout le parcours d'achat. C'est ce mode qui a été vérifié ici.

Fichiers concernés

Fichier	Rôle
<code>src/Services/PaymentService.php</code>	<code>createIntent()</code> (créer l'intention) et <code>verify()</code> (revérifier le paiement)
<code>src/Controllers/Front/CheckoutController.php</code>	Orchestration du tunnel de commande
<code>public/assets/js/checkout.js</code>	Logique côté navigateur (confirmation de la carte)
<code>resources/views/front/checkout.php</code>	Page de paiement (champ Stripe ou message démo)
<code>src/Core/Response.php</code>	Autorise les scripts Stripe (politique de sécurité CSP)

Comment le tester avec un vrai compte Stripe

1. Créer un compte Stripe (gratuit) et récupérer les **clés de test** (`pk_test_...` et `sk_test_...`).
2. Les renseigner dans le fichier `.env` (ou `.env.docker`).
3. Sur la page de paiement, utiliser la carte de test **4242 4242 4242 4242**, une date future et n'importe quel CVC.

[!] Cette vérification « réelle » n'a pas pu être faite automatiquement car elle nécessite tes clés Stripe et une saisie dans le navigateur. Le **code est correct** (méthode standard Stripe) et le **mode simulé est validé**.

2. Codes de réduction (promotions)

L'objectif

Permettre à un client de saisir un **code promo** dans son panier pour obtenir une remise (en pourcentage ou en montant fixe), et permettre aux administrateurs de **créer et gérer** ces codes.

Comment ça marche

- Le client saisit un code dans le panier -> le système vérifie qu'il est **valide** (existe, actif, non expiré, pas au-delà de sa limite d'utilisation).
- La remise est calculée et affichée : **sous-total -> remise -> total**.
- Au paiement, le montant débité est le **total après remise**, et la commande garde en mémoire le code utilisé et le montant de la remise.
- Chaque utilisation d'un code **incrmente son compteur** (pour gérer les limites).

Trois codes de démonstration sont fournis :

Code	Effet
BIENVENUE10	-10 %
CYNA25	-25 % (limité à 100 utilisations)
SECURE50	-50 €

Exemple vérifié

Panier de 499 € + code `CYNA25` (-25 %) -> remise 124,75 € -> **total 374,25 €**. La commande enregistrée contient bien `code_reduction = CYNA25` et `remise_centimes = 12475`.

Côté administrateur

Un nouveau menu « **Promotions** » dans le back-office permet de : - voir tous les codes (avec leur nombre d'utilisations) ; - créer un code (pourcentage ou montant fixe, expiration, limite d'usage) ; - activer/désactiver ou supprimer un code.

Fichiers concernés

Fichier	Rôle
<code>database/migrations/2026_07_promotions.sql</code>	Crée la table <code>codes_reduction</code> + 2 colonnes sur les commandes
<code>src/Services/Promotion.php</code>	Calcul de la remise + validation (avec tests unitaires)

Fichier	Rôle
<code>src/Repositories/PromotionRepository.php</code>	Accès base (lire/créer/supprimer les codes)
<code>src/Controllers/Front/CartController.php</code>	Appliquer / retirer un code au panier
<code>src/Controllers/Admin/PromotionController.php</code>	Gestion back-office
<code>resources/views/front/cart.php</code>	Champ code + ligne de remise
<code>resources/views/admin/promotions/index.php</code>	Écran d'administration
<code>tests/Unit/PromotionTest.php</code>	13 tests du calcul de remise

3. Support en temps réel + filtres (application Java)

L'objectif

Les messages envoyés via le formulaire de contact du site sont déjà enregistrés en base. Il fallait que l'**application Java** les affiche **en temps réel** et propose des **filtres**.

Ce qui a été ajouté

- **Temps réel** : la liste se met à jour **automatiquement toutes les 5 secondes** dès qu'un nouveau message arrive — sans avoir à relancer l'application. La sélection en cours est préservée (pas de « saut » gênant).
- **Filtres** :
 - par **statut** : Tous / Nouveau / Lu / Archivé ;
 - par **recherche texte** : sur l'e-mail, le sujet ou le contenu du message ;
 - un bouton **Réinitialiser** et le **tri** par colonne.
- Un indicateur vert « ● **Temps réel** » signale l'actualisation automatique.

Comment c'est fait (en une phrase)

Un minuteur (`Timer`) vérifie discrètement si la base a changé (nombre de messages ou dernier identifiant) ; si oui, il recharge la liste. Les filtres utilisent un `TableRowSorter` qui masque/affiche les lignes sans toucher à la base.

Fichier concerné

`src/main/java/com/cyna/admin/ui/panels/SupportPanel.java`

4. Export CSV des commandes (back-office)

L'objectif

Fournir un **rapport exportable** des commandes, ouvrable dans Excel.

Ce qui a été ajouté

Un bouton « **Exporter (CSV)** » sur la liste des commandes du back-office. Le fichier généré contient : numéro de facture, date, e-mail, statut, sous-total, code promo, remise et total. Il **respecte le filtre de statut** actif et inclut un « BOM UTF-8 » pour que les accents s'affichent correctement dans Excel.

Fichiers concernés

Fichier	Rôle
<code>src/Controllers/Admin/OrderController.php</code>	Méthode <code>export()</code> qui construit le CSV
<code>src/Repositories/OrderRepository.php</code>	<code>allForExport()</code> récupère les commandes
<code>resources/views/admin/orders/index.php</code>	Bouton d'export

Ce qui a été vérifié (avec Docker)

Contrôle	Résultat
Tests unitaires PHP (PHPUnit)	[OK] 77 tests
Tests unitaires Java (JUnit)	[OK] 10 tests, build réussi
Application d'un code promo (réel)	[OK] 499 € -> 374,25 €
Commande complète (Stripe simulé + promo)	[OK] commande créée
Export CSV	[OK] 1003 commandes exportées

Ce qui reste à décider

- **Détection de fraude** (fonctionnalité optionnelle) : non implémentée car elle demande de **définir des règles** (ex. bloquer si trop de commandes du même e-mail en peu de temps, ou au-delà d'un certain montant). À préciser.
- **Test Stripe réel** : nécessite tes **clés Stripe de test** et un navigateur.

Glossaire

Terme	Explication simple
PaymentIntent	« Intention de paiement » créée côté Stripe : un objet qui représente un paiement d'un montant donné, que le client va confirmer.
client_secret	Jeton temporaire que Stripe renvoie pour autoriser le navigateur à confirmer le paiement, sans exposer les clés secrètes.
Mode simulé	Repli automatique quand aucune clé Stripe n'est configurée : le paiement est joué « pour de faux » afin de tester le parcours.
CSRF	Jeton de sécurité qui accompagne chaque formulaire pour empêcher qu'un site tiers déclenche une action à ta place.
Migration	Script SQL qui fait évoluer la base de données (ici : ajouter la table des codes et deux colonnes aux commandes) sans perdre les données existantes.
BOM UTF-8	Petit marqueur en début de fichier CSV qui indique à Excel d'afficher correctement les accents.
TableRowSorter	Composant Java qui trie et filtre les lignes d'un tableau à l'écran, sans modifier les données sous-jacentes.